Data Structure and Algorithm, Spring 2026

Homework 4

RED CORRECTION: 18:00:00, May 21, 2026

BLUE CORRECTION: 23:00:00, May 28, 2026

Due: 13:00:00, Wednesday, June 3, 2026

TA E-mail: dsa_ta@csie.ntu.edu.tw

Rules and Instructions

- Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.
- Discussions on course materials and homework solutions are encouraged. But you should write the final solutions alone and understand them fully. Books, notes, and Internet resources (including chatGPT) can be consulted, but not copied from.
- Since everyone needs to write the final solutions alone, there is **absolutely no need to lend your homework solutions and/or source codes to your classmates at any time.** In order to maximize the level of fairness in this class, lending and borrowing homework solutions are both regarded as dishonest behaviors and will be punished according to the honesty policy.
- We will detect and penalize plagiarism with the criteria of **line-by-line similarity** in any part of your code/writing, when compared with the code/writing from any student or any external resource. Exceptions will be given to those showing **time-stamped records** that the solution comes from a meaningful interactive discussion with human and/or AI tools, **rather than copying from** others.
- In Homework 4, the problem set contains 4 problems. The set is divided into two parts, the non-programming part (Problems 1, 2) and the programming part (Problems 3, 4).

- For your solutions in the non-programming part, unless otherwise noted, there are two common requirements:
  - Your answer should be as simple as possible. For instance, $\Theta(\frac{1}{n} + \lg(n^{541}) + 8763n^2)$ should be written as $\Theta(n^2)$.
  - Your answer should also come with a *brief* explanation. The word *brief* means "very short." An example of a *brief* explanation is as follows.

    Sample problem: Write down the time complexity for this algorithm using the Θ notation along with an expression of n.

    HELLO-WORLD(n)

    ```
    1  for i = 1..2n
    2      for j = 1..2n
    3          print("Hello world")
    ```

    Sample solution: The iteration counts of the two loops are $2n$ and $2n$ respectively, so there are $4n^2$ iterations. Hence, the time complexity is $\Theta(n^2)$. □

- For problems in the non-programming part, you should combine your solutions in *one* PDF file. Your file should generally be legible. Your solution must be as simple as possible. At the TAs' discretion, solutions which are too complicated can be penalized or even regarded as incorrect. If you would like to use any theorem which is not mentioned in the classes, please include its proof in your solution.

- The PDF file for the non-programming part should be submitted to Gradescope as instructed, and you should use Gradescope to tag the pages that correspond to each subproblem to facilitate the TAs' grading. **A 10% penalty applies if no pages are tagged, and a 5% penalty if at least one (but not all) correct pages are tagged.**

- For problems in the programming part, you should write your code in C programming language, and then submit the code via the *DSA Judge+*: https://dsa.csie.ntu.edu.tw/. Each day, you can submit up to 10 times for each problem. The judge system will compile your code with

```
gcc [id].c -O2 -lm -std=c17
```

- For debugging in the programming part, we strongly suggest you produce your own test-cases with programs and/or use tools like `gdb` or compile with flag `-fsanitize=address` to help you solve issues like illegal memory usage. For `gdb`, you are strongly encouraged to use compile flag `-g` to preserve symbols after compiling.

  Note: For students who use IDE (VScode, DevC++...) you can modify the compile command in the settings/preference section. Below are some examples:

  - `gcc [id].c -O2 -lm -std=c17 -fsanitize=address`
    `# Works on Unix-like OS, but possibly not M1/M2`
  - `gcc [id].c -O2 -lm -std=c17 -g # use it with gdb`

- The score of the part that is submitted after the deadline will get some penalties according to the following rule:

$$\textit{Late Score} = \max\left(\left(\frac{5 \times 86400 - \textit{Delay Time(sec.)}}{5 \times 86400}\right) \times \textit{Original Score}, 0\right)$$

  As mentioned at the beginning of the class, you have **eight gold medals** this semester, each of which can be used to extend the deadline of **one** programming problem by **24 hours**. You can choose how to allocate the medals around the end of the semester.

- We use the gender-neutral pronouns *ze/zir* throughout this problem set.

- If you have questions about this homework, please go to the Discord channel and discuss the issues with your classmates. This is *strongly preferred* because it will provide everyone with a more interactive learning experience. If you really need email clarifications, you are encouraged to include the following tag in the subject of your email:

  - Please include the following tag in the subject of your email: `"[HW4.Px]"`, specifying the problem where you have questions. For example, `"[HW4.P2] Is k in subproblem 5 an integer?"`. Adding the tag allows the TAs to track the status of each email and to provide faster responses to you.

– If you want to provide your code segments to us as part of your question, please upload it to Gist or similar platforms and provide the link. Please remember to protect your uploaded materials with proper permission settings. Screenshots or code segments directly included in the email are discouraged and may not be reviewed.

Non-Programming Part

Problem 1 - Daltonism-Safe Algorithm (100 pts)

1. (10 pts) At Don't Sleep Anymore University, a student named Hsiao-Ming (HM) was writing zir DSA homework late one night. The homework required zir to draw a Red-Black tree, but ze made a horrifying discovery —ze had color weakness and could not distinguish the red nodes from the black ones. Staring at the figures in the textbook, every node looked identical to zir, making it impossible to learn the Red-Black tree structure directly.

   Fortunately, from a previous lecture, HM recalled that we have learned about another fundamental balanced search tree data structure: B-trees. Surprisingly, these two seemingly different structures are closely related. In fact, a Red-Black tree can structurally represent any B-tree with a minimum degree of $t = 2$. The two trees are equivalent in the following sense: they store the same keys in the same order and both achieve $O(\log n)$ time complexity for search, insertion, and deletion operations.

   This gave HM a great idea: by first studying the B-tree structure (which uses no colors), ze can derive the corresponding Red-Black tree — including the colors of each node — purely from the structural transformation rules, without ever needing to rely on zir ability to distinguish colors.

   Here are the rules of the transformation from a B-tree ($t = 2$) to a Red-Black tree. We separate the transformation into three types:

   - If a B-tree node has one key, it corresponds to a black node in the Red-Black tree.
   - If a B-tree node has two keys, it corresponds to a black node with a red left child in the Red-Black tree.
   - If a B-tree node has three keys, it corresponds to a black node with a red left child and a red right child in the Red-Black tree.

   When performing the transformation, remember that the child nodes in the original B-tree must also be properly connected in the corresponding Red-Black tree structure. Think about how the in-order traversal of both trees should remain consistent.

   Note that other valid transformation rules also exist, but when strictly following the rules given above, the resulting Red-Black tree is uniquely determined by the input B-

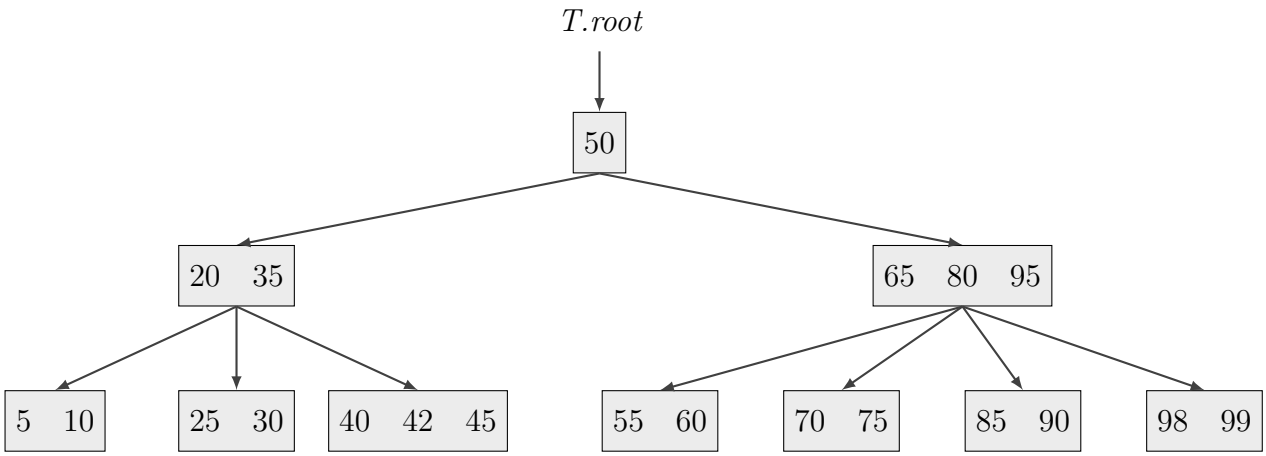


Figure 1.1: A B-tree with $t = 2$

tree. Following the rules above, please convert the B-tree ($t = 2$) in Figure 1.1 to its corresponding Red-Black tree and draw the resulting tree.

2. (25pt) While working on zir homework, HM found that there exists a relationship between the balancing mechanisms of B-trees and Red-Black trees. Consider the B-tree ($t = 2$) structure shown in Figure 1.2, in which a parent with two keys (three children) is connected to one of its children that has three keys (four children). Each letter denotes a subtree.

   During a top-down B-Tree-Insert(), the algorithm descends from the root and is about to enter the $[15, 20, 25]$ node. Because this node is full, the algorithm performs a *pre-emptive split* before descending. Since the split happens before the new key reaches a leaf, you only need to consider the structural transformation, not the key insertion. The pseudo code functions are provided below for your reference. These are identical to the ones provided in the lecture slides.

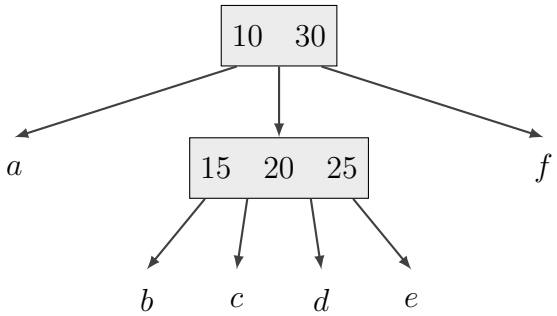


Figure 1.2: A B-tree ($t = 2$) structure where a parent with two keys is connected to a child with three keys

(a) (5pt) Please draw the B-tree immediately after the preemptive split.

(b) (5pt) Convert your answer in (a) to its equivalent Red-Black tree using the transformation rules from problem 1.1.

(c) (5pt) Convert the B-tree in Figure 1.2 to its equivalent Red-Black tree using the transformation rules from problem 1.1.

(d) (10pt) Starting from your answer in (c), apply a sequence of recolorings and rotations to obtain your answer in (b). Draw every intermediate Red-Black tree after each individual recoloring or rotation.

B-Tree procedures from the lecture slides

```
B-Tree-Insert(T, k)
 1  r = T.root
 2  if r.n == 2t − 1
 3        s = Allocate-Node()
 4        T.root = s
 5        s.leaf = FALSE
 6        s.n = 0
 7        s.c_1 = r
 8        B-Tree-Split-Child(s, 1)
 9        B-Tree-Insert-Nonfull(s, k)
10  else
11        B-Tree-Insert-Nonfull(r, k)
```

```
B-Tree-Insert-Nonfull(x, k)
 1  i = x.n
 2  if x.leaf
 3       while i ≥ 1 and k < x.key_i
 4            x.key_{i+1} = x.key_i
 5            i = i − 1
 6       x.key_{i+1} = k
 7       x.n = x.n + 1
 8       Disk-Write(x)
 9  else
10       while i ≥ 1 and k < x.key_i
11            i = i − 1
12       i = i + 1
13       Disk-Read(x.c_i)
14       if x.c_i.n == 2t − 1
15            B-Tree-Split-Child(x, i)
16            if k > x.key_i
17                 i = i + 1
18       B-Tree-Insert-Nonfull(x.c_i, k)
```

```
B-Tree-Split-Child(x, i)
 1  z = Allocate-Node()
 2  y = x.c_i
 3  z.leaf = y.leaf
 4  z.n = t − 1
 5  for j = 1 to t − 1
 6        z.key_j = y.key_{j+t}
 7  if y.leaf == FALSE
 8        for j = 1 to t
 9              z.c_j = y.c_{j+t}
10  y.n = t − 1
11  for j = x.n + 1 downto i + 1
12        x.c_{j+1} = x.c_j
13  x.c_{i+1} = z
14  for j = x.n downto i
15        x.key_{j+1} = x.key_j
16  x.key_i = y.key_t
17  x.n = x.n + 1
18  Disk-Write(y)
19  Disk-Write(z)
20  Disk-Write(x)
```

3. (20pt) After the midterm, HM wants to look up how many students scored within a certain grade range — but the Office of Academic Affairs displays the results in red, which HM cannot distinguish due to zir color weakness. Ze decides to write zir own query system.

   Formally, given a B-tree with minimum degree t that stores n integer keys, design an algorithm that, given a query range $[l, r]$ where l and r are integers, returns the number of keys k in the tree such that $l \leq k \leq r$. Your counting algorithm should answer each query in $O(t \log_t n)$ time and use at most $O(n)$ total extra space (including any fields you add to B-tree nodes).

   You may modify the provided B-tree procedures to support your algorithm; if you do, your modifications must not asymptotically increase the time complexity of those procedures. Please provide your full solution — including any modifications and your new procedure(s) — in pseudo code, using at most 30 lines total, and briefly explain why it is correct and

meets the complexity requirements.

4. (25pt) The B-tree implementation introduced in the lecture uses *preemptive split / merge*. In this implementation, a check is performed *before* inserting or deleting a key to determine whether the maximum or minimum degree rule would be violated after the insertion or deletion. If so, a split or a merge is performed *before* the key insertion or deletion, respectively. In contrast, *non-preemptive split / merge* performs the same check *after* a key has been inserted or deleted. A split or a merge is only performed when the check discovers that the maximum or minimum degree rule is violated.

   In this problem, we ask you to compose the pseudo code of B-Tree-Insert-2(T, k), a function that inserts key k into an existing B-tree T, but uses non-preemptive split.

   You can freely use the two functions Split-Child-2(x, i) and Split-Root-2(T) without providing any explanation in your solution. Note that these are slightly different from the ones introduced in the lecture and in the textbook. These functions are splitting a node with $2t$ keys, which has one more key than the maximum number of keys allowed $(2t - 1)$. They perform the operations illustrated in Figure 1.3 and with the details as follows:

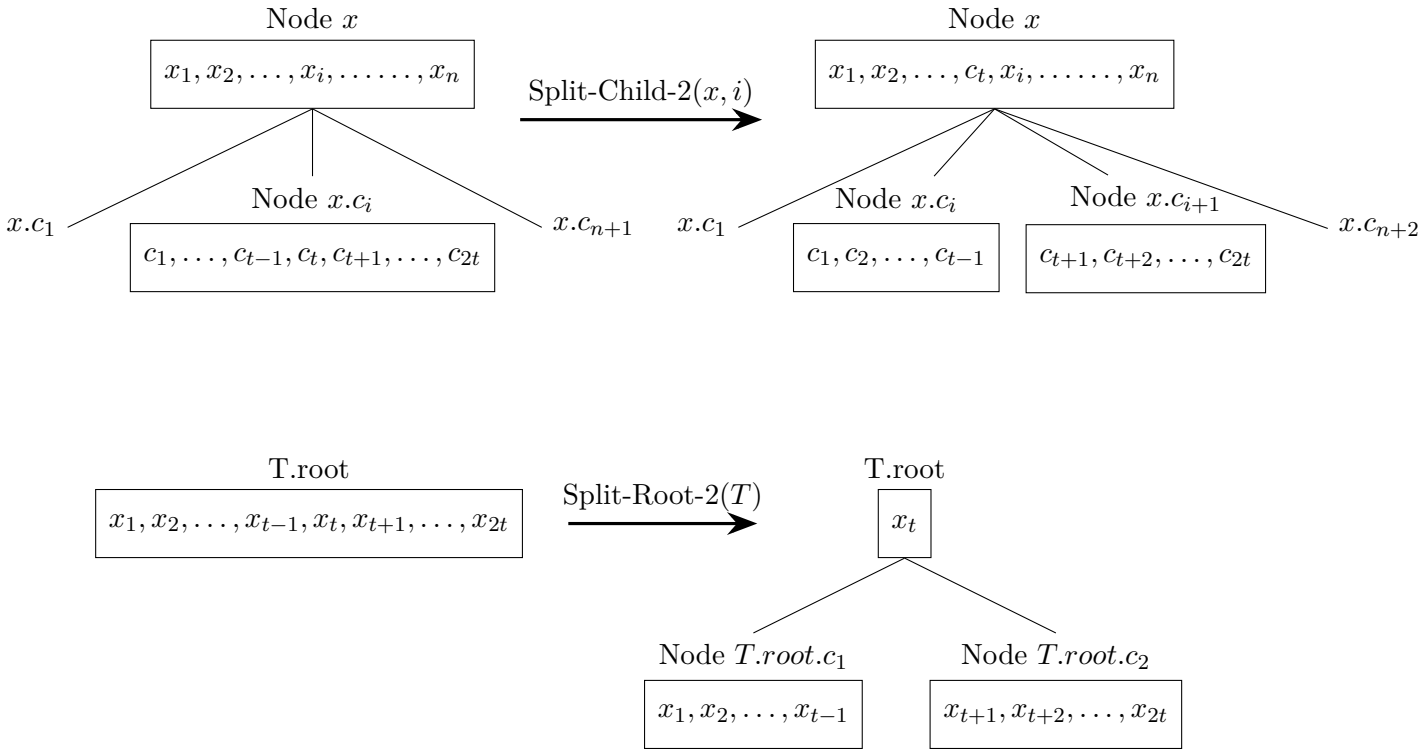


Figure 1.3: B-Tree Split-Child-2 and Split-Root-2 operations

- Split-Child-2(x, i): Split x's child node $x.\, c_i$ (which has exactly $2t$ keys) into two

neighboring child nodes, node $x.c_i$ with $t-1$ keys and node $x.c_{i+1}$ with t keys, and insert the t-th key in original $x.c_i$ node into the i-th position of the list of the keys of x. Note that after inserting the t-th key, node x might have $2t$ keys.

- SPLIT-ROOT-2(T): Split root node $T.root$ with exactly $2t$ keys into two nodes, node $T.root.c_1$ with $t-1$ keys and node $T.root.c_2$ with t keys. Then, create a new node with only one key x_t, the t-th key in the original root node. Make this new node the new $T.root$, with $T.root.c_1$ and $T.root.c_2$ as its first and second child nodes.

Please provide a brief explanation to help the grading TA to better understand your pseudo code.

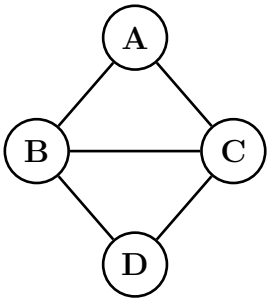
5. (20pt) In a typical computer system, accessing the memory (both read and write) is much faster than accessing the hard disk, even when the latter is a solid-state drive. However, the capacity of the memory is usually significantly smaller than that of the hard disk. Therefore, the data are typically stored on the hard disk, and are only moved to memory when frequent processing is expected. Assume all B-tree data is initially stored on the hard disk. Also assume that disk hardware allows you to choose the size of a disk block arbitrarily, but the time it takes to read a disk block is $a + b \cdot t$, where a and b are given constants and t is the minimum degree of a B-tree using blocks of the selected size.

   Determine a t value such that the Disk-Read time during the search process in the B-tree is minimized, given that $a = 1$ millisecond, $b = 64$ microseconds, and the number of keys in the tree $n = 1024$. Here, as an approximation, you can assume that all nodes have the same maximum number of keys for a chosen t value, although in reality some nodes might have fewer keys due to the fixed value of $n = 1024$. In addition, describe how you determine this t value.

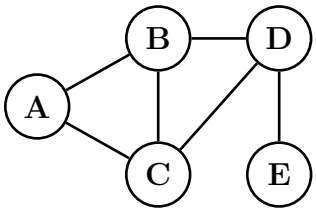
   (Hint: What is the tree height for a given t value?)

Problem 2 - Don't Sleep Anymore! (100 pts)

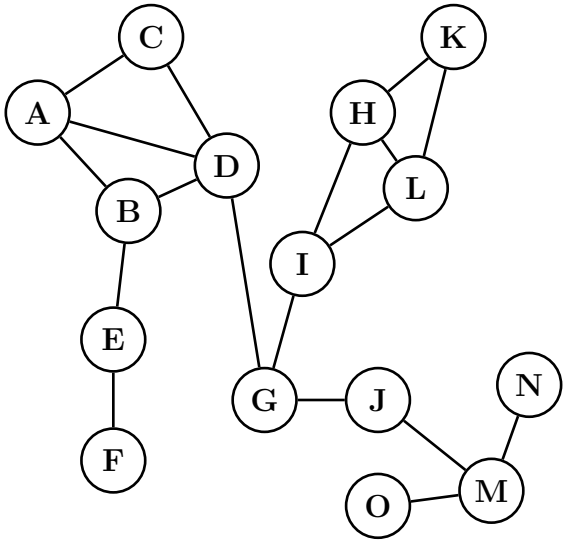
1. (10 pts) Hsiao-Ming is currently taking the required course DSA at Don't Sleep Anymore University. Having just learned BFS and DFS, ze saw Figure 2.1 and really wants to perform these searches on it. However, since ze hasn't quite mastered them yet, ze needs your help. Please answer the following questions regarding Figure 2.1 to help Hsiao-Ming.



(a) Graph A



(b) Graph B



(c) Graph C

Figure 2.1: Three graphs for the BFS/DFS counting problem.

(a) (5 pts) In this problem, we ask you to give the number of distinct **BFS** sequences starting from node A. Give the answer as an integer and **without explanation**. For example, for Graph A in Figure 2.1(a), there are two possible BFS sequences, $A \to B \to C \to D$ and $A \to C \to B \to D$. Therefore, the answer is 2. In a similar manner, please derive the answers for (i) Graph B in Figure 2.1(b); (ii) Graph C in Figure 2.1(c).

(b) (5 pts) In this problem, we ask you to give the number of distinct **DFS** sequences starting from node A. Give the answer as an integer and **without explanation**. For example, for Graph A in Figure 2.1(a), there are four possible DFS sequences, $A \to B \to C \to D$, $A \to B \to D \to C$, $A \to C \to B \to D$, and $A \to C \to D \to B$. Therefore, the answer is 4. In a similar manner, please derive the answers for (i) Graph B in Figure 2.1(b); (ii) Graph C in Figure 2.1(c).

2. (15 pts) Hsiao-Ming was scrolling through zir phone during zir DSA class and came across a statement online. Please help zir determine whether this statement is correct.

Statement 2.2

Let $G = (V, E)$ be an undirected graph and A be the adjacency matrix that represents G. For any positive integer n and for all $u, v \in V$, the number of paths of length n from u to v is given by the entry $A^n[u][v]$, where $A^n = \underbrace{A \times A \times A \times \cdots \times A}_{n}$, and the operator "$\times$" represents matrix multiplication. Note: According to the definition provided in the lectures, a path in this context allows for the repetition of vertices and edges.

- If you believe the statement above is correct, please prove it using mathematical induction.
- If you believe the statement above is incorrect, please provide a counterexample.

3. (25 pts) You and Hsiao-Ming both studied very hard in high school and were finally admitted to your dream school – Don't Sleep Anymore University. Upon enrollment, you receive a curriculum list consisting of the n courses, numbered 1 to n, that you must complete to graduate. You also receive an integer list $T = [T_1, T_2, T_3, \ldots, T_n]$, where each T_i is the number of months required to complete course i. Since some courses have prerequisites, you also receive a prerequisite list $R = [R_1, R_2, R_3, \ldots, R_m]$ of length m, where each $R_j = (u_j, v_j)$ indicates that you must finish course u_j before you can start course v_j. The course prerequisite graph is guaranteed to be a DAG.
To thoroughly embody the school motto of Don't Sleep Anymore University – "DON'T SLEEP ANYMORE!" – you may have any number of courses in progress at the same time, with no cap on credit load. A course can begin as soon as all of its prerequisites have been completed.
Given n, T, m, R, your task is to design an $O(n + m)$-time and $O(n + m)$-extra-space algorithm to compute the earliest month you can graduate (i.e., the earliest month by which you have completed all courses).

Please provide your algorithm in pseudo code (at most 40 lines) and briefly justify its correctness and complexity.

4. (25 pts) Hsiao-Ming is also a big fan of chess, and after learning DSA, ze has come up with two DSA-related chess challenges! Ze has shared the first challenge with you, and it's up to you to solve it.

   At the beginning of the game, you are given an $n \times m$ chessboard with only two white knights, K_a and K_b, placed at $(1, 1)$ and (n, m), respectively.

   On each day, you must move both knights exactly once, where each move follows the standard rules for a knight in chess, provided below. The goal of the challenge is to have both knights land on the same square at the end of some day, after both have completed their moves.

   However, every day, before the two knights make their moves, an evil person will place a white pawn on some square of the board in an attempt to stop you. You are also given a list of locations where the pawns will be placed from day 1 to day $n \times m$: $P = [P_1, P_2, P_3, \ldots, P_{n \times m}]$, where $P_t = (x_t, y_t)$ represents the position of the pawn placed on day t, with x_t and y_t denoting the row and column, respectively. It is guaranteed that $P_i \neq P_j$ whenever $i \neq j$, i.e., the pawns are placed on distinct squares, implying that the board will be completely filled with pawns on day $n \times m$.

   Some additional rules:

   - You cannot move a knight to a square occupied by a pawn. Furthermore, pawns cannot be removed (i.e., captured) once placed on the board.
   - You cannot move the pawns placed by the evil person.
   - The evil person is allowed to place a pawn on a square currently occupied by one of your knights; in that case, the knight is removed from the board, making it impossible to reach the goal. Since you already know where the pawns will be placed, this can be avoided.
   - You can assume that, for the given board dimensions n and m, if no pawns were placed, there would exist a solution in which the two knights meet on some day after both have moved. However, depending on the order in which the evil person places the pawns, it may be impossible for the two knights to meet.

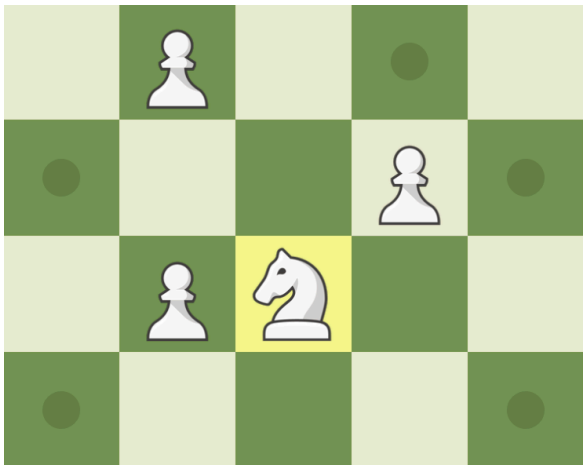
   Given n, m, P, please answer the following:

   (a) (10 pts) A student claims that, in any optimal solution (i.e., one that achieves the earliest meeting day), neither knight ever visits the same cell more than once on

its way to the meeting square. Use a concrete example to illustrate why this claim must be true.

(b) (15 pts) Using the characterization established in part (a), design an $O(nm)$-time and $O(nm)$-extra-space algorithm to compute the earliest day on which the two knights can land on the same square. If the two knights can never land on the same square, please return -1.
Please provide your algorithm in pseudo code (at most 50 lines) and briefly justify its correctness and complexity.

- The movement rule of the knight: each move follows an uppercase "L" shape – moving two squares in one direction and then moving one square in a perpendicular direction.
  - The figure below shows a 4×5 chessboard. The positions to which the knight can move are the 5 locations marked with grey dots:



  - If you are still unsure how the knight moves, you can refer to our friendly TAs or this video: Knight – ChessKid

5. (25 pts) (This question has nothing to do with the previous problem, it is just that the story is related. You can start with this question even if you haven't written the previous yet.)
Next, Hsiao-Ming told you the other challenge and you have to solve it, too.

At the beginning of the game, you are given an $n \times m$ chessboard. A single white rook is provided but is not yet placed on the board.

On each day, you must place the rook on some square $(a, 1)$ in the leftmost column and then move it to some square (b, m) in the rightmost column, where $1 \leq a, b \leq n$. Each

move must follow the standard rules for a rook in chess, provided below, and you may move the rook any number of times on a given day. After completing all moves on that day, you must remove the rook from the board.

However, every day, before you move the rook, an evil person will place a white pawn on some square of the board in an attempt to stop you. You are also given a list of locations where the pawns will be placed from day 1 to day $n \times m$: $P = [P_1, P_2, P_3, \ldots, P_{n\times m}]$, where $P_t = (x_t, y_t)$ represents the position of the pawn placed on day t, with x_t and y_t denoting the row and column, respectively. It is guaranteed that $P_i \neq P_j$ whenever $i \neq j$, i.e., the pawns are placed on distinct squares, implying that the board will be completely filled with pawns on day $n \times m$.

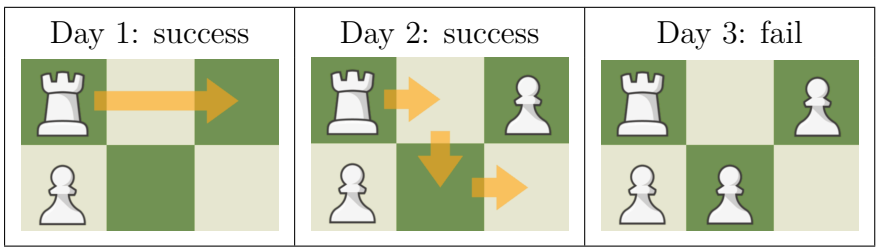
Some additional rules:

- You cannot place the rook on a square occupied by a pawn. Furthermore, pawns cannot be removed (i.e., captured) once placed on the board.
- The rook cannot jump over squares blocked by pawns.
- You cannot move the pawns placed by the evil person.
- You may assume that $n \geq 1$ and $m \geq 2$.

Given n, m, P, your task is to design an $O(nm \cdot \log(nm))$-time and $O(nm)$-extra-space algorithm to calculate the last day T that the challenge can be completed (starting from day $T + 1$, the rook will no longer be able to move from the leftmost column to the rightmost column). If the challenge is infeasible even on day 1, please return 0.

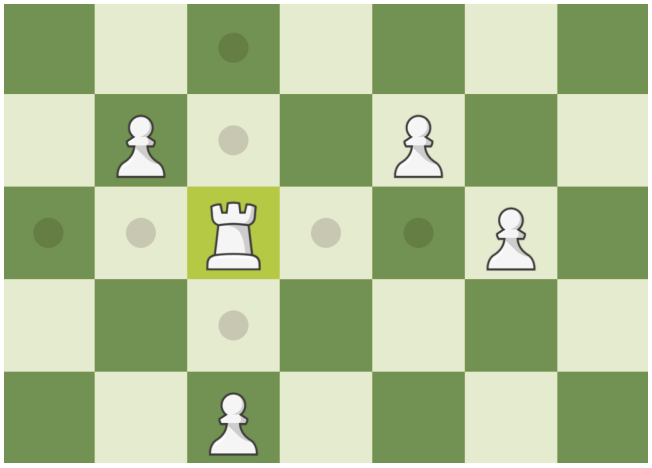
Please provide your algorithm in pseudo code (at most 50 lines) and briefly justify its correctness and complexity.

Example: For $n = 2, m = 3$, and $P = [(2, 1), (1, 3), (2, 2), (1, 1), (1, 2), (2, 3)]$, the challenge will start to fail from Day 3. Therefore, the last day that the challenge can be successfully completed is Day 2, making the answer 2:



- Rook's movement rule: In each move, a rook can move to any square within the same row or column as itself. It cannot jump over any pawns, and it cannot move outside the boundaries of the chessboard.

– The figure below shows a 5×7 chessboard. The positions to which the rook can move are the locations marked with grey dots:



– If you are still unsure how the rook moves, you can refer to our friendly TAs or this video: Rook – ChessKid

Problem 3 - Dynamic Stock Acquisitions (100 pts)

Problem Description

After graduating from Don't Sleep University (the rival of Don't Sleep Anymore University), Hsiao-Ting (HT) has become a newcomer to society and noticed that the competition between firms is cutthroat. There are N firms in this country, numbered 1 to N. Each firm i starts with an initial valuation of s_i. Initially, every firm operates independently, that is, firm i alone forms the i-th **group**. Let v_i denote the valuation of firm i.

As an outstanding DSU alumnus, HT will record exactly one of the following four events over the next Q days:

- `Merge x y` - The group containing firm x and the group containing firm y combine into a single group. If x and y are already in the same group, do nothing.
- `Revalue p w` - Every firm in the same group as firm p has its valuation increased by w.
- `Antitrust p` - Regulatory authorities force firm p to be separated from its group. Firm p leaves its current group and forms a new group by itself, retaining its current valuation. The remaining firms stay together as one group. If the group containing firm p has only one firm, ignore this event.
- `Audit p` - Output the valuation of p and the total valuation of the group containing firm p.

After processing all Q events, output the final valuation of the firms, $v_1, v_2, \ldots, v_N$, separated by spaces. Since this problem revolves around groups that merge and split, it is natural to organize the firms using a Disjoint-Set Union (DSU) forest, with each firm represented by a node and each group by a tree. Standard DSU handles `Merge` directly, and the size of a group can be tracked by storing it on the root.

Let G be a group of firms. To avoid $\Theta(|G|)$ per `Revalue` call on group G, potentially very slow with a large group, we introduce a technique called the **lazy tag**: rather than storing each firm's valuation directly, we attach a value t_i to every node i in the DSU forest – including non-root nodes – and we maintain the invariant

$$v_i = \sum_{j \in \text{path}(i)} t_j,$$

where path(i) is the sequence of nodes from node i to the root of its DSU tree, inclusive. Initially every firm is its own root, so we set $t_i = s_i$ for each i. The key observation is that

operations affecting *every member* of a group can now be performed by writing to the root alone: if r is the root of p's group and we add w to t_r, then for every descendant i of r, the root r still lies on path(i), so firm i's valuation under the invariant automatically increases by w. Reading firm i's current valuation, conversely, is done by walking from i up to the root and summing the tags along the way.

Merging two groups requires a small adjustment to keep the invariant consistent. Suppose we merge groups with roots r_a and r_b by attaching r_b as a child of r_a. Every firm previously in r_b's group now has t_{r_a} added to its path sum, even though their actual valuations should not change. To compensate, we offset t_{r_b} at the moment of merging:

$$\begin{cases} \text{parent}(r_b) = r_a \\ t_{r_b} = t_{r_b} - t_{r_a} \end{cases}$$

After this adjustment, every firm in the old r_b-group keeps the same path sum as before, while future writes to t_{r_a} correctly affect the entire merged group.

You are encouraged to use this lazy-tag formulation, paired with **union by rank** (or union by size), to keep operations efficient.

Input

The first line contains two integers N and Q, representing the number of firms and the total number of operations, respectively.

The second line contains N space-separated positive integers $s_1, s_2, \ldots, s_N$, indicating the initial valuation of firm 1 through firm N.

The following Q lines each describe a single operation. Every operation begins with a keyword followed by its specific arguments, separated by single spaces. The formats for these operations are as follows:

- `Merge x y`
- `Revalue p w`
- `Antitrust p`
- `Audit p`

Output

For every `Audit` operation, output a single line `<p_value> <group_value>`, where `<p_value>` is the current valuation of p, and `<group_value>` is the total valuation of the group that p belongs to.

After all operations, output one final line with N space-separated integers: the valuation of firm 1 through firm N.

Constraints

- $1 \leq N, Q \leq 10^6$
- $1 \leq w, s_i \leq 10^9$
- The valuation of each firm throughout the process does not exceed 10^{15}.
- All firm indices are valid (between 1 and N).

Subtasks

Subtask 1 (15 pts)

- $N, Q \leq 2 \times 10^3$

Subtask 2 (20 pts)

- Only `Merge`, `Revalue`, and `Audit` operations.

Subtask 3 (25 pts)

- Only `Merge`, `Antitrust`, and `Audit` operations.

Subtask 4 (40 pts)

- No additional constraints.

Sample Testcases

Sample Input 1

```
5 7
10 20 30 40 50
Merge 1 2
Revalue 1 5
Merge 2 3
Audit 1
Antitrust 1
Audit 1
Audit 2
```

Sample Output 1

```
15 70
15 15
25 55
15 25 30 40 50
```

Sample Input 2

```
6 10
5 5 5 5 5 5
Merge 1 2
Merge 3 4
Merge 1 3
Revalue 2 10
Audit 1
Antitrust 1
Audit 1
Audit 3
Antitrust 3
Audit 3
```

Sample Output 2

```
15 60
15 15
15 45
15 15
15 15 15 15 5 5
```

Sample Input 3

```
4 8
100 50 75 25
Merge 1 2
Merge 3 4
Revalue 1 200
Audit 1
Audit 3
Merge 1 3
Antitrust 1
Audit 1
```

Sample Output 3

```
300 550
75 100
300 300
300 250 75 25
```

Problem 4 - Diligent Seminar Accountant (100 pts)

Problem Description

Story

Years have passed, and HM has now become the Deputy Vice President for Academic Affairs at Don't Sleep Anymore University, while HT has returned to Don't Sleep University as a professor specializing in DSA. They became friends in an academic exchange between the two universities. Recently, HM invited HT to be the *Splendid Exchange Nonresident Supervisor of Engineering in Information*, affectionately abbreviated as *SENSEI*, of Don't Sleep Anymore Science School, the prestigious high school affiliated with Don't Sleep Anymore University.

Don't Sleep Anymore Science School is a school known for its free and open atmosphere. Students in various clubs engage in diverse activities, ranging from game development and robotics to exploring paranormal phenomena. The student council, known as Seminar, oversees administrative tasks, implements policies, and manages the school's budget.

Hayase-Yuu (HY), the biracial child of HM, is the accountant of Seminar, responsible for budget allocation. The largest portion of expenditures comes from club funding applications. There are N clubs in the school, and every day, different clubs submit funding requests to Seminar. To handle these requests, Seminar has established a dynamic priority system.

Because of continuous new applications and approvals, the relative rankings of the clubs change frequently. HY previously managed this using the school's **Hyper**computer, but it was recently destroyed by a stray projectile from the Engineering Department. HY is now distressed and needs to rebuild this financial management system. Unfortunately, both HY and zir parent, HM, have color weakness and are therefore unable to help. This is why HM reached out to HT to invite zir as *SENSEI* —to help HY with the data structure problem ze faced.

HY, the accountant of Seminar, maintains this priority system as follows. The system is implemented via a self-balancing data structure with colors.

Each club is represented by a unique ID i and a multiset S_i of application amounts[1]. The total application sum of club i is defined as $\sum_{s \in S_i} s$. HY ranks the clubs in descending order of this sum, breaking ties by smaller ID first. Let $r_1, r_2, \ldots, r_N$ denote the IDs of the clubs in this ranked order; that is, r_K is the ID of the K-th ranked club.

[1]Unlike a set, a multiset may contain the same element more than once.

HY's system supports three kinds of operations: adding funds, adding an application, and approving an application.

- **Adding funds**: Seminar earns additional funds of amount m, and the total budget M increases by m.
- **Adding an application**: Club i receives a new application with amount x, which is added to its multiset S_i.
- **Approving an application**: HY distributes funds from the current total budget M by selecting a number K and reviewing the K-th ranked club. From that club, ze approves the largest amount $L \in S_{r_K}$ such that $L \leq M$. Once approved, L is removed from S_{r_K}, and the total budget becomes $M - L$. If S_{r_K} is empty or all elements of S_{r_K} exceed M, no application is approved.

Input

Integers on the same line are separated by a single space.

The first line contains three integers Q, N, M, representing the number of events, the number of clubs, and the initial total budget of Seminar, respectively.

The next N lines describe the initial state. The i-th line starts with an integer R_i, representing the number of initial applications from club i. This is followed by R_i integers $x_{i_1}, x_{i_2}, \ldots, x_{i_{R_i}}$, representing the amounts of these initial applications.

The following Q lines each describe an event:

- `1 C x`: Add an application of amount x to club C.
- `2 m`: Add funds of amount m to Seminar's budget.
- `3 K`: Approve an application by reviewing the K-th ranked club.

Output

For each event, output the result on a new line. Multiple integers on the same line are separated by a single space.

- Event 1: Output the updated total application sum $\sum_{s \in S_C} s$ of club C.
- Event 2: Output the updated total budget M.
- Event 3: Output two integers r_K and L, representing the ID of the reviewed club and the approved amount, respectively. If no application is approved, $L = 0$.

Constraints

- $1 \leq Q \leq 5 \times 10^5$
- $1 \leq N \leq 10^5$

- $1 \leq M \leq 10^9$
- $0 \leq \sum_{i=1}^{N} R_i \leq 5 \times 10^5$
- $1 \leq C, K \leq N$
- $1 \leq x_{i_k}, x, m \leq 10^9$
- Throughout the process, M and the total application sum of any club never exceed 10^{15}.

Subtasks

Subtask 1 (10 pts)

- $1 \leq N, Q \leq 2000$
- $0 \leq \sum R_i \leq 2000$

Subtask 2 (25 pts)

- No additional constraints on N, Q.
- $R_i = 1$ for all i.
- Only events 2 and 3 occur.

Subtask 3 (25 pts)

- No additional constraints on R_i.
- Includes all event types (1, 2, 3).
- For every event 3, $K = 1$.

Subtask 4 (40 pts)

- No additional constraints.

Sample Testcases

Sample Input 1

```
6 3 100
2 20 50
1 30
2 10 80
3 2
1 2 60
3 1
2 80
3 3
3 3
```

Sample Output 1

```
1 50
90
2 30
100
1 20
1 0
```

Sample Input 2

```
6 5 60
1 40
1 70
1 10
1 40
1 90
3 3
3 2
2 80
3 2
3 1
3 3
```

Sample Output 2

```
1 40
2 0
100
2 70
5 0
3 10
```

Sample Input 3

```
5 4 200
2 150 60
1 300
2 40 50
0
3 1
2 150
3 1
1 3 120
3 1
```

Sample Output 3

```
2 0
350
2 300
210
1 0
```